

Lab 17: AI for Data Processing: Data Cleaning and Preprocessing Scripts

Task 1: Traffic Sensor Data Cleaning

Prompt:

1. Load traffic sensor dataset that contains missing values and incorrect data entries in some columns.
2. Clean vehicle count and average speed columns by filling missing values properly.
3. Remove duplicate rows so each sensor reading is unique in the dataset.
4. Convert timestamp column into proper datetime format for analysis.
5. Remove invalid values like negative speed and standardize intersection names to uppercase.

Code:

```
import pandas as pd

df = pd.read_csv('traffic.csv')

df['vehicle_count'].fillna(df['vehicle_count'].mean(), inplace=True)
df['avg_speed'].fillna(df['avg_speed'].mean(), inplace=True)

df.drop_duplicates(inplace=True)

df['timestamp'] = pd.to_datetime(df['timestamp'])

df = df[(df['vehicle_count'] >= 0) & (df['avg_speed'] >= 0)]

df['intersection'] = df['intersection'].str.upper()

print(df)
```

Output:

```
... sensor_id intersection      timestamp  vehicle_count  avg_speed
0      1          A  2024-01-01 10:00:00         20.00         40
1      2          B  2024-01-01 10:05:00         26.25         35
3      4          C  2024-01-01 10:15:00         30.00         50
/tmp/ipykernel_24243/1343786795.py:5: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series th
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which w
```

Explanation:

1. Missing values are filled using mean.
2. Duplicate rows are removed.
3. Invalid negative values are filtered out.
4. Data is formatted properly for analysis.

Task 2: Movie User Behavior Data

Prompt:

1. Load movie platform dataset and convert login date into datetime format.
2. Extract new features like day of week and weekend or weekday.
3. Encode subscription type and device type into numeric values.
4. Normalize watch time values between 0 and 1.
5. Detect and remove extreme outliers in watch time.

Code:

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

df = pd.read_csv('movie.csv')

if 'login_date' not in df.columns:
    print(f"Error: 'login_date' column not found in the DataFrame. Available columns are: {df.columns.tolist()}")
else:
    df['login_date'] = pd.to_datetime(df['login_date'])

    df['day'] = df['login_date'].dt.day_name()
    df['weekend'] = df['login_date'].dt.dayofweek >= 5

    df = pd.get_dummies(df, columns=['subscription_type', 'device_type'])

    scaler = MinMaxScaler()
    df['watch_time'] = scaler.fit_transform(df[['watch_time']])

    Q1 = df['watch_time'].quantile(0.25)
    Q3 = df['watch_time'].quantile(0.75)
    IQR = Q3 - Q1
    df = df[(df['watch_time'] >= Q1 - 1.5*IQR) & (df['watch_time'] <= Q3 + 1.5*IQR)]

    print(df)
```

Output:

```
python3 /tmp/ipykernel_24243/1343786795.py:5: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series, and its current value is being overwritten
df['watch_time'] = scaler.fit_transform(df[['watch_time']])
```

	sensor_id	intersection	timestamp	vehicle_count	avg_speed
0	1	A	2024-01-01 10:00:00	20.00	40
1	2	B	2024-01-01 10:05:00	26.25	35
3	4	C	2024-01-01 10:15:00	30.00	50

Explanation:

1. Date column is converted properly.
2. New features like day and weekend are added.
3. Categorical data is encoded.
- 4.** Outliers are removed for better analysis.

Task 3: Space Telemetry Data

Prompt:

1. Load satellite telemetry dataset with missing and noisy values.
2. Fill missing numerical values like temperature and fuel level.
3. Encode system status into numeric format.
4. Remove unrealistic values like negative fuel levels.
5. Scale data and split into training and testing sets.

Code:

```
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

df = pd.read_csv('space.csv')

df.fillna(df.mean(numeric_only=True), inplace=True)

df['system_status'] = df['system_status'].map({'OK':1, 'FAIL':0})

df = df[df['fuel_level'] >= 0]

scaler = StandardScaler()
df[['temperature_reading', 'fuel_level', 'signal_strength']] = scaler.fit_transform(
    df[['temperature_reading', 'fuel_level', 'signal_strength']]
)

X_train, X_test = train_test_split(df, test_size=0.2)

print(X_train)
```

Output:

	sensor_id	intersection	timestamp	vehicle_count	avg_speed
0	1	a	2024-01-01 10:00	20.0	40
1	2	b	2024-01-01 10:05	NaN	35
2	3	A	2024-01-01 10:10	25.0	-10
3	4	c	2024-01-01 10:15	30.0	50
4	4	c	2024-01-01 10:15	30.0	50

Explanation:

1. Missing values are filled using mean.
2. Status column is encoded.
3. Invalid data is removed.
4. Data is scaled and split for ML use.

Task 4: Gaming Data Cleaning

Prompt:

1. Load gaming dataset that contains duplicate records and missing values.
2. Remove duplicate player records from the dataset.
3. Fill missing match duration values using average.
4. Encode categorical columns like region and win status.
5. Standardize rank values and create score per minute feature.

Code:

```
import pandas as pd

df = pd.read_csv('game.csv')

df.drop_duplicates(inplace=True)

df['match_duration'].fillna(df['match_duration'].mean(), inplace=True)

df['rank'] = df['rank'].str.upper()

df = pd.get_dummies(df, columns=['region', 'win_status'])

df['score_per_minute'] = df['player_score'] / df['match_duration']

print(df)
```

Output:

```
...   player_id  player_score  match_duration  rank  region_asia \
0         1         200         30.0  BRONZE      True
1         2         300         27.5  BRONZE     False
2         3         250         25.0  SILVER     True

   region_europe  win_status_No  win_status_Yes  score_per_minute
0         False         False         True         6.666667
1          True          True         False        10.909091
2         False         False         True        10.000000
/tmp/ipykernel_211/4173502423.py:7: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series consisting of multiple rows and columns. Please use .loc[row_index, col_index] = value instead.
```

Explanation:

1. Duplicate records are removed.
2. Missing values are filled.
3. Rank values are standardized.
4. New feature is created for analysis.

Task 5: Climate Dataset Preparation

Prompt:

1. Load climate dataset with missing values and inconsistent country names.
2. Standardize country names into uppercase format.
3. Fill missing CO2 emissions and temperature values.
4. Encode extreme event column into numeric format.
5. Normalize data and generate summary report.

Code:

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

df = pd.read_csv('climate.csv')

missing_before = df.isnull().sum()

df['country'] = df['country'].str.upper()

df.fillna(df.mean(numeric_only=True), inplace=True)

df['extreme_event'] = df['extreme_event'].map({'Yes':1, 'No':0})

scaler = MinMaxScaler()
df[['CO2_emissions', 'avg_temperature']] = scaler.fit_transform(
    df[['CO2_emissions', 'avg_temperature']]
)

missing_after = df.isnull().sum()

print(df)

print("Missing Before:\n", missing_before)
print("Missing After:\n", missing_after)
print("Unique countries:", df['country'].nunique())
```

Output:

```
...   country  CO2_emissions  avg_temperature  year  extreme_event
0    INDIA      0.000000      0.714286  2020             1
1    INDIA      0.641975      1.000000  2021             0
2     USA      0.925926      0.571429  2020             1
3     USA      1.000000      0.000000  2021             0
Missing Before:
country          0
CO2_emissions    1
avg_temperature  1
year             0
extreme_event    0
dtype: int64
Missing After:
country          0
CO2_emissions    0
avg_temperature  0
year             0
extreme_event    0
dtype: int64
Unique countries: 2
```

Explanation:

1. Country names are standardized.
2. Missing values are filled properly.
3. Data is encoded and normalized.
4. Summary report shows improvement.